

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)
Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Обработка строк

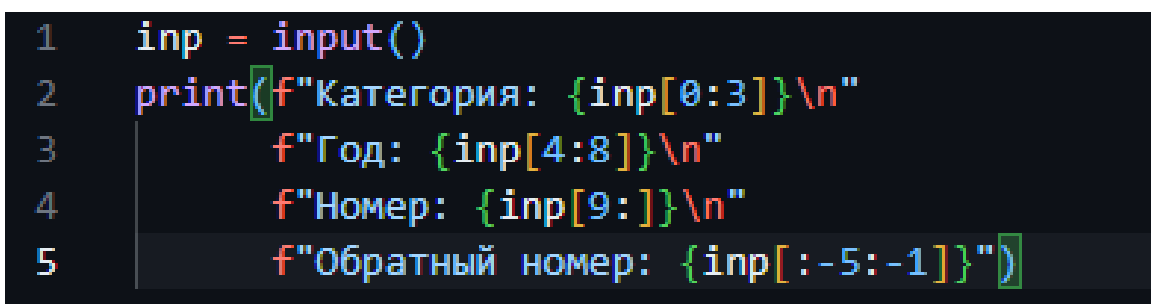
Студент		
гр. БИН-26-1	_____	У. Т. Чан-Ван-Ю
Преподаватель	_____	М. В. Водяницкий

1 Выполнение работы

1.1 Задание 1. Разбор кода документа

Первое задание предполагает разбор строки, содержащей код документа в формате AAA-NNNN-NNNN, где первая часть представляет собой три заглавные латинские буквы категории, вторая — четыре цифры года, третья — четыре цифры номера. Необходимо извлечь категорию, год и номер, используя операцию среза, при этом сохранив ведущие нули. Дополнительно требуется вывести номер в обратном порядке.

Исходный код программы с результатами её выполнения приведён на рисунке 1.



```

1  inp = input()
2  print(f"Категория: {inp[0:3]}\n"
3      f"Год: {inp[4:8]}\n"
4      f"Номер: {inp[9:]}\n"
5      f"Обратный номер: {inp[::-5:-1]}")

```

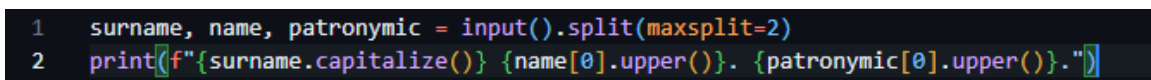
Рисунок 1 – Исходный код и результат выполнения первого задания

Для извлечения отдельных частей строки применяются срезы: символы с индексами от 0 до 3 соответствуют категории, с 4 до 8 — году, а начиная с 9-го — номеру документа. Обратный порядок номера получается с помощью среза с отрицательным шагом. Вывод результатов осуществляется функцией `print()` с использованием f-строк и символа переноса строки `\n`.

1.2 Задание 2. Форматирование имени

Во втором задании пользователь вводит фамилию, имя и отчество единой строкой с произвольным регистром символов. Между частями ФИО может находиться произвольное количество пробелов, включая возможные пробелы по краям строки. Требуется вывести фамилию, записанную с заглавной буквы, а также инициалы имени и отчества в верхнем регистре.

Исходный код программы с результатами её выполнения приведён на рисунке 2.



```

1  surname, name, patronymic = input().split(maxsplit=2)
2  print(f"{surname.capitalize()} {name[0].upper()}. {patronymic[0].upper()}")

```

Рисунок 2 – Исходный код и результат выполнения второго задания

Разделение строки на отдельные части выполняется методом `split()` с параметром `maxsplit=2`, который ограничивает число разбиений двумя, благодаря чему отчество, даже содержащее пробелы, сохраняется целиком. Форматирование осуществляется методами `capitalize()` для фамилии и `upper()` для первых букв имени и отчества.

1.3 Задание 3. Очистка номера телефона

Третье задание заключается в обработке номера телефона, введённого пользователем в формате +7 (NNN) NNN-NN-NN, где N обозначает цифру от 0 до 9. Необходимо удалить все служебные символы: знак плюса, пробелы, круглые скобки и дефисы, оставив только цифры.

Исходный код программы с результатами её выполнения приведён на рисунке 3.

```
1 num = input()
2 print(num.replace("+", "").replace("(", "").replace(")", "").replace("-", "").replace(" ", ""))
```

Рисунок 3 – Исходный код и результат выполнения третьего задания

Для удаления ненужных символов используется строковый метод `replace()`, вызываемый последовательно для каждого из удаляемых символов. Результатом является строка, содержащая исключительно цифры телефонного номера.

1.4 Задание 4. Разбор записи о поезде

Четвёртое задание требует обработать строку, содержащую информацию о поезде в формате номер;откуда;куда;время;цена, где все пять полей разделены символом точки с запятой. Необходимо разделить строку на отдельные значения и вывести их в оформленном виде, при этом цену следует отобразить с двумя цифрами после десятичной точки.

Исходный код программы с результатами её выполнения приведён на рисунке 4.

```
train, city_dispatch, city_destination, time_dispatch, price = input().split(";", maxsplit=4)
print(f"Поезд: {train}\n"
      f"Маршрут: {city_dispatch} - {city_destination}\n"
      f"Отправление: {time_dispatch}\n"
      f"Цена: {(float(price)).2f} руб")
```

Рисунок 4 – Исходный код и результат выполнения четвёртого задания

Разделение строки на поля выполняется методом `split()` с разделителем `;` и параметром `maxsplit=4`, что позволяет корректно обработать названия городов, содержащие пробелы. Для форматирования цены используется преобразование в тип `float` и спецификатор формата `.2f`, обеспечивающий вывод ровно двух знаков после десятичной точки.

1.5 Задание 5. Проверка идентификатора

В пятом задании пользователь вводит произвольную строку, которая может быть пустой или содержать пробелы. Требуется определить её длину и выполнить четыре проверки: состоит ли строка исключительно из букв, исключительно из цифр, является ли буквенно-цифровой, а также содержит ли символ дефиса. Для первых трёх проверок необходимо использовать соответствующие строковые методы.

Исходный код программы с результатами её выполнения приведён на рисунке 5.

```
s = input()
print(f"Длина: {len(s)}\n"
      f"Только буквы: {s.isalpha()}\n"
      f"Только цифры: {s.isdigit()}\n"
      f"Буквенно-цифровая: {s.isalnum()}\n"
      f"Содержит дефис: {'-' in s}")
```

Рисунок 5 – Исходный код и результат выполнения пятого задания

Длина строки определяется встроенной функцией `len()`. Проверки выполняются строковыми методами `isalpha()`, `isdigit()` и `isalnum()` соответственно. Наличие дефиса определяется оператором принадлежности `in`. Все результаты выводятся в формате, соответствующем условию задания.

1.6 Задание 6. Формирование пути

Шестое задание предполагает обработку строки, содержащей ровно три части пути, разделённые запятыми без пробелов. Необходимо разделить строку на отдельные части и соединить их символом прямой косой черты, формируя таким образом путь в стиле операционных систем семейства Unix.

Исходный код программы с результатами её выполнения приведён на рисунке 6.

```
1 print("/".join(input().split(",")))
```

Рисунок 6 – Исходный код и результат выполнения шестого задания

Разделение исходной строки осуществляется методом `split()` с разделителем в виде запятой. Полученный список частей передаётся в метод `join()`, вызываемый для строки-разделителя `/`, что обеспечивает корректное формирование итогового пути.

2 Вывод

В ходе выполнения лабораторной работы были изучены основные операции и методы обработки строк в языке программирования Python. На практике освоены получение подстрок с помощью срезов, форматирование строк через f-строки, разделение и соединение строк методами `split()` и `join()`, замена символов методом `replace()`, а также использование строковых методов `isalpha()`, `isdigit()`, `isalnum()`, `capitalize()` и `upper()` для анализа и преобразования текста. Полученные навыки позволяют эффективно работать со строковыми данными, что является важной частью практического программирования.